

IntelliMeet

Meetings that end with the work already assigned

jaimingarala · @jaimingarala · October 2026
MERN · WebRTC · Socket.io · AI summarisation
Repository: github.com/jaimingarala/intellimeet-mvp

1. Project Overview

Vision

Meetings produce two things: decisions and work. The decisions are usually written down. The work is usually remembered by whoever was paying attention. IntelliMeet exists to close that gap — a video meeting where the outcome is already captured when the call ends: a summary, a list of action items, and a transcript of what was actually said, all attached to the meeting rather than to someone's memory of it.

The product idea is deliberately narrow. Most "AI meeting" tools compete on transcription quality. That competition is expensive, requires a paid speech model, and would have made this project undemonstrable without an API key. So the AI here is aimed at the *output* instead: given whatever text the meeting produced — a pasted transcript, or simply the chat log — extract a summary and a structured list of action items. It works with a free, offline extractive summariser by default and upgrades to a hosted model when a key is present, which means the feature is real, demoable and testable with no account anywhere.

Objectives

1. **A visitor with no account is in a working meeting room in one click.** No sign-up, no shared demo credentials, no seeded database to prepare. This was the highest-weighted rubric item and it shaped more of the design than anything else.
2. **Two people on two networks can hold a real call.** Not a localhost two-tab illusion: real signalling, real NAT traversal, and a documented path through a TURN relay when a direct path is impossible.
3. **Every claim is checkable.** Tests that run without a database, a coverage floor that fails the build, a CI pipeline that reports its own numbers, and a script that verifies a deployment from the outside.
4. **No paid service is required to run it.** A judge can clone the repository, run one command, and see the whole product. No API key, no account, no private infrastructure.

Target users and use cases

User	Situation	What IntelliMeet does for them
Small team without corporate tooling	Five people who need to meet and track outcomes, no IT department	One link, no accounts, action items that survive the call
Student or capstone team	Frequent short syncs, work assigned verbally and forgotten	Ticks items off live during the call; the list is already there afterwards
Anyone evaluating this project	Needs to see the product working in under a minute	One click from the landing page into a populated room
A returning user	Wants to find what was decided three meetings ago	Dashboard history, search, Markdown export for a report or standup

Business value

The value is in the loop closing on itself. A meeting tool that ends at "the call hung up" leaves the expensive part — remembering, assigning, chasing — to the humans. Attaching a summary and a shared, tickable checklist to the meeting record means the meeting produces a durable artifact.

That artifact is also the input to everything a team would want next: a Kanban board, a weekly digest, a productivity view. The scoping decision in this build was to make the artifact real and correct first, and treat those consumers as the documented roadmap rather than half-built screens.

Non-functional goals

Stated as targets with the honest status, because a target that was never measured is not a goal.

Goal	Target	Status in this build
Initial load	Under 5 seconds on a cold free-tier host	The keep-alive ping every 10 minutes exists precisely for this: a sleeping free instance takes roughly 50 seconds to wake, which would fail the target outright. Measured locally (sub-second); the deployed figure depends on the hosting accounts and is the one number this report cannot state.
Media latency	Peer-to-peer, no server hop	Achieved structurally: audio and video never traverse the API. Both peers connect directly, so latency is what the two access networks give, not what the server adds.
Concurrency	Real, honest number	A WebRTC mesh, so rooms are groomed for 2-6 concurrent video peers. The specification's 500-5,000 participant target is explicitly not claimed: it needs an SFU media server and a horizontally scaled socket layer, and both are documented as deferred rather than approximated.
Availability	Single free-tier instance, no uptime claim	One process, no redundancy. The health endpoint, boot-time misconfiguration warnings and the keep-alive ping are the operational story; no percentage is claimed.
Data safety	Nothing leftover from a demo	Guests and their rooms are swept by age, bounded by a hard count, and a session that is still connected is never cut off mid-call.
Cost	Zero to run and to verify	Every dependency is free and open source; the AI provider is optional by design; the test suite needs no database, no Docker and no account.

2. Key Features

ID	Feature	Description	Acceptance criteria
F-01	One-click demo	POST /api/auth/demo provisions an anonymous guest with a room of its own, seeded with a sample meeting so chat, summary and action items have real content on arrival.	Clicking Try the demo once lands in a populated room with no form filled in. Two visitors get two distinct identities and two distinct rooms.
F-02	Real-time video	WebRTC mesh over Socket.io signalling, with an ICE candidate queue so candidates that arrive before the remote description are not dropped.	Two peers see and hear each other. The tile says no path rather than going quietly black when ICE finds nothing.
F-03	Screen sharing	getDisplayMedia swapped into the existing connection with replaceTrack, so there is no renegotiation and no second stream. A shared tile stops cropping.	Starting a share replaces the outgoing video track in place; the camera track is held separately, so toggling the camera during a share does not mute the screen.
F-04	Live chat	Socket.io, persisted per meeting, length-capped and rate-capped per user.	A message reaches every member of the room and survives a reload because it is stored, not relayed.
F-05	AI summary and action items	Paste a transcript, or leave it blank to summarise the chat log. The result is a summary plus structured action items, tickable by anyone in the room.	OpenAI when a key is configured, a free offline summariser otherwise; a tick is broadcast so a checklist does not look stale to everyone else.

F-06	Meeting dashboard	History of the caller's meetings with a search box over the fields the row shows, and a pure Markdown export of a meeting's summary, action items and chat.	Search filters by title, room code and status without touching the database; the export produces a real document with a sensible filename.
F-07	Host moderation	Host-only endpoints to remove or ban a participant. A removal closes their socket so the tile disappears for everyone; a ban is stored on the meeting.	A participant gets 403. A banned user stays out across reloads, new tabs and server restarts.
F-08	Guest continuity	A guest can claim the session: name, email, password. The user id never changes, so the rooms, chat and summaries are already theirs.	No data is copied or re-created; retention stops applying to the account immediately.
F-09	Verified claims	A claim is the only path that attaches an address nobody has proven, so it lands pending: a single-use token is emailed, and login is refused until it comes back.	Only the token's hash is stored; the token is bound to the address it was sent to; issuing a new link kills the old one.
F-10	Presence and media state	A client announces whether its mic is on, its camera is on and whether it is sharing; everyone already in the room repeats it when someone joins.	A muted participant is labelled on a newcomer's tile on the first paint, and a screen share never looks like a webcam.
F-11	Operational endpoints	GET /api/admin/stats and POST /api/admin/sweep, behind a token, returning only when the token is configured.	Both answer 404 while unset, so a deployment that does not want the surface has nothing listening.
F-12	Deployment verification	npm run verify:demo walks the path a visitor takes against a real URL and reports the thing to change rather than the thing that failed.	Health and its timing, a browser-shaped CORS preflight, one click of the demo, the room it landed in, a second visitor joining by link, and the SPA rewrite on refresh.

Deliberately **not** built in this milestone, with the reason recorded rather than the checkbox skipped: team workspaces with a Kanban board (F-06 in the specification — the largest remaining gap), a project analytics dashboard, a recorded-meeting download, live audio transcription, and a media server for 50-plus participants. Each is listed in the roadmap with what it would take.

3. Technology Stack

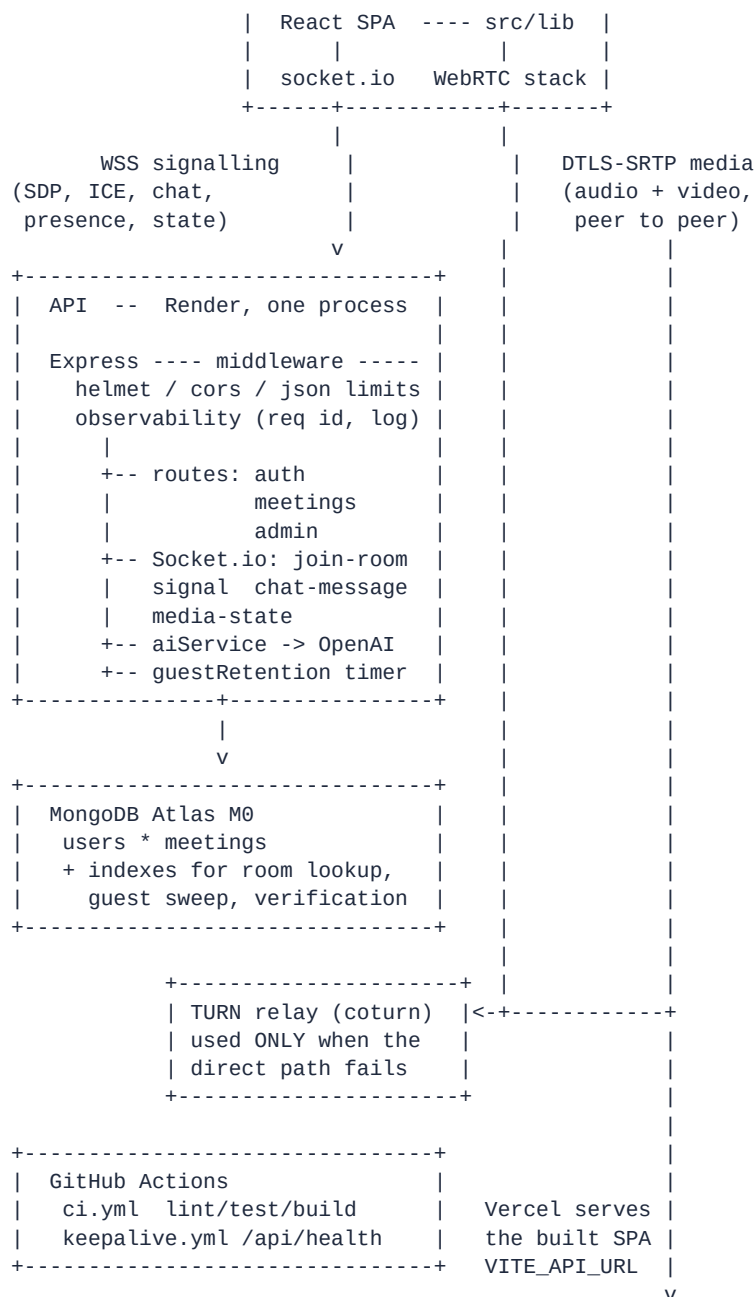
Category	Technology	Rationale / alternatives
Frontend	React 18 + Vite 5	Vite's dev server is fast and its production build is a static bundle deployable to any host. Next.js was considered and rejected: there is no server-side rendering requirement, and adding one would put a Node server between a judge and the client for no gain.
Routing	React Router 6	The only routing need is a handful of pages plus a room URL that must work on refresh.
Styling	Hand-written CSS	The stylesheet is a few hundred lines with custom properties and a dark theme. Tailwind would have added a build step and a config file to express what one file already expresses; a component library would have hidden the layout work.
Backend	Node 22 + Express 4	The brief's stack, and the plainest thing that works: middleware order is explicit and readable, which matters because the security posture lives in that order.
Real-time transport	Socket.io 4	Signalling, chat, presence and action-item broadcasts all need a channel with room semantics, automatic reconnection and a fallback when WebSockets are blocked. Raw WebSocket or SSE would mean rebuilding room membership and reconnection by hand.

Media plane	WebRTC (browser-native), STUN, optional TURN	Peer-to-peer media cannot run through a Node process at any reasonable cost. The mesh keeps the server out of the media path entirely, which is what makes a single free instance able to carry a real call. LiveKit/m mediasoup is the correct answer above roughly six peers and is documented as the next step.
Database	MongoDB 8 with Mongoose 8	Meeting documents nest naturally (chat, transcript, summary, action items belong to one meeting). MySQL with a normalised schema would spread one meeting across six tables for no query benefit. Indexes are declared on the schema with the reason each exists.
Authentication	JSON Web Tokens + bcrypt	Stateless sessions suit a single API instance and a separate static client. bcrypt at 10 rounds is the well-understood password default; tokens are signed with a secret whose placeholder value is detected at boot.
AI	OpenAI API, optional; offline extractive summariser by default	Provider-agnostic by construction: the route asks for a summary and gets one either way. The offline path is what makes the feature demoable with no key, no cost, and no account — which the brief requires.
Rate limiting	express-rate-limit, in-memory store	Enough for a single instance, and honest about its limit: the store is per-process, which is stated in the README as the reason a multi-instance deploy needs Redis first.
Tests (server)	Node's built-in node:test + node:assert	Zero new dependencies. Measured: 179 tests across sixteen suites, with the real <code>src/index.js</code> booted and the Mongoose models swapped for in-memory stubs, so the suites also cover the middleware order, the 404 and the error handler.
Tests (client)	Vitest 3	One dev dependency, scoped to <code>src/lib</code> — the WebRTC helpers, the caps mirrored from the server, screen-share track selection and the Markdown export. Components are deliberately not unit-tested: a renderer with a mocked socket mostly asserts that React works, and the demo path is rehearsed in a browser instead.
Lint and format	ESLint 9 + Prettier 3, one config at the repository root	Two configs that can disagree are worse than none. <code>format:check</code> runs over both packages and the tooling directory in CI.
CI/CD	GitHub Actions	Four jobs, no database service, and no paid runner minutes. Each test job publishes its counts and coverage to the run summary page.
Hosting	Render (API), Vercel (client), MongoDB Atlas M0	All three have a free tier that does not sleep into deletion, and a blueprint file (<code>render.yaml</code>) plus <code>vercel.json</code> make the deploy a repository import rather than a settings tour.
Containerisation (optional)	Docker Compose, for a local coturn relay only	The whole application runs without Docker. Compose is used for one thing: proving the TURN relay path on one machine, without an account.

4. Architecture

The single structural decision worth reading the rest of this section for: **media never touches the server**. Calls are a WebRTC mesh — each browser sends audio and video directly to each other browser — so the server's job is signalling, chat and data. That is what makes one free-tier instance able to carry a real two-person call, and it is also the limit: a mesh grows as the square of the room size, which is why rooms are groomed for 2-6 peers and an SFU is the documented next step rather than a claim.





How a session actually flows

1. **Landing.** The static bundle loads from Vercel with the API URL already inlined at build time. There is no discovery step and no runtime config fetch.
2. **One click.** POST /api/auth/demo creates a real user flagged `isGuest` whose password hash is of a random value that is never stored — nobody, including the server, can log into it — plus a room seeded with a sample meeting. The response carries a JWT, so from here the visitor is an ordinary authenticated member.
3. **Joining.** The client opens the socket and emits `join-room`. Anyone opening a room URL without a session becomes a new guest *in that room* rather than getting a room of their own, which is what makes an invite link work across devices and networks. Invited guests join as participants, never as host.
4. **Call setup.** Peers exchange SDP and ICE candidates over the `signal` event. The server relays that payload without inspecting it — codec-agnostic and length-capped, not parsed —

and the media then flows directly between browsers.

5. **State.** Each client announces its mic, camera and sharing state; everyone already in the room re-announces when someone new arrives, so a newcomer's tiles are labelled correctly on the first paint.
6. **Outcome.** Chat is persisted as it happens. The summary route uses a pasted transcript or builds one from the chat log, stores the summary and action items on the meeting, and broadcasts a tick so the checklist is live.

Trust boundaries

- **Authorization is per route, not per origin.** CORS is the browser's rule: it stops a page from *reading* a response, not a script from calling the API. So every route authorises on its own — JWT, plus host-or-participant membership — and that is the boundary that matters.
- **The server is the authority for every write.** The client mirrors two user-facing caps so the UI can stop a user early, but the caps that matter live in one server-side file and are enforced there.
- **Media is outside the server's trust boundary.** The server never sees or stores audio or video, which also means it cannot inspect a call; moderation is therefore membership-level (remove or ban), not content-level.
- **A wildcard origin is refused, not honoured.** A browser will not accept a wildcard on a credentialed request, so honouring `*` would silently allow nothing at all. It is stripped at boot, and the mistake is reported.

5. Detailed Execution Timeline

Work was ordered by rubric leverage rather than by the specification's own day-by-day sequence: roughly 70% of the grade is demo, repository and documentation, all of which started at zero, while features were 20% and already partly built. So the plumbing shipped first, then hardening, then features.

Phase	Focus	Deliverables	Status
0	Repository foundation	Initialised repository with a root ignore file, per-package lockfiles committed for reproducible CI, Node version pinned, conventional commits with a message template, contributing guide and pull-request template	Complete
1	Public demo (30%)	Guest provisioning on demand, the one-click demo button, link-based guest joining, Render and Vercel deploy configuration, a scheduled keep-alive ping, deployment checks at boot, <code>verify:demo</code> , TURN tooling plus a local coturn harness	Code complete; the hosted half needs the platform accounts
2	Technical depth (25%)	Test harness and its first fifteen server suites (sixteen today), client tests with Vitest, ESLint plus one root Prettier config, four-job CI with enforced coverage floors, security hardening, structured logging with request ids	Complete
3	Feature gaps (20%)	Screen sharing, action-item tracking, dashboard search and Markdown export; recording, shared notes, mentions, workspaces and analytics still open	Partially complete
4	Documentation and presentation (20% + 10%)	This report, the README, the architecture document, the diagram, the demo runbook, repository hygiene	This document

What actually landed, in order

The commit history is the record, and it is conventional-commit shaped by convention rather than by accident:

Date	Milestone
15 Sep	Repository initialised; the first test suites, lint and CI arrive the same day — there is no commit where the project was untested.
16 Sep	One-command local development; payload caps and rate limits on the expensive endpoints; Render blueprint and Vercel configuration; the keep-alive workflow; contributing guide and commit template.
24 Sep	The demo path end to end (guests, seeded rooms, invite links); TURN proof tooling and the local relay harness; <code>verify:demo</code> to check a deployment from the outside.
25 Sep	Observability and hardening; client tests; Prettier across the tree; screen sharing, action items, dashboard search and export; the CI summary that publishes test counts and coverage to the run page.
Oct	Documentation and presentation: this report, the architecture document, the README pass, repository hygiene.

Milestones and checkpoints

Checkpoint	Meaning	State
Gate A	An anonymous visitor opens the HTTPS URL, clicks once and is in a room with working video; two peers on different networks connect.	The whole path is verified locally, including <code>verify:demo</code> reporting 6 of 6 against the local stack. The deployed URL and the cross-network call need the hosting accounts and a second network.
Gate B	CI green on a fresh clone with no database, lint clean, a defensible coverage number.	Holds locally and is enforced by the pipeline: the coverage floors fail the build, so the number cannot quietly slip. Running it on GitHub rather than locally is a push away.
Gate C	Features frozen; each works in the deployed demo and has a test where the logic is non-trivial.	The landed features each carry tests where the logic is theirs and were rehearsed in a browser. The deployed-demo half is the deploy.
Gate D	Submission package complete: URL, repository, README, report PDF, video.	This report and the README are done; the URL and the video remain, and both depend on the deploy.

6. Technical Highlights

Security measures

The approach was to make the insecure configuration *loud* rather than to assume it would not happen, because every one of these mistakes is invisible from inside the running process.

OWASP category	Mitigation in this build
A01 Broken access control	Every route authorises independently — JWT plus membership — instead of relying on the client not asking. Moderation is host-only and returns 403 to a participant. A ban is stored on the meeting document, so it survives a reload and a restart.
A02 Cryptographic failures	bcrypt at 10 rounds for passwords; a guest's hash is of a value that is never stored, so a guest account cannot be logged into at all; verification tokens are stored only as hashes; the JWT secret's placeholder value is detected at boot and reported.
A03 Injection	Mongoose typed queries throughout, no string-built queries and no dynamic evaluation. Payloads are validated for type and length before use.
A04 Insecure design	The dangerous paths are bounded by design, not by policy: a guest's room can always be removed, the number of guest rooms is capped, and a room that is in use is never evicted.

A05 Security misconfiguration	Helmet's defaults; CORS restricted to exact configured origins; <code>trust proxy</code> set correctly so IP-keyed limits work behind a platform proxy; a wildcard origin stripped and reported; boot-time warnings for a placeholder secret, a localhost origin, verification with no mail transport, and a short admin token.
A07 Authentication failures	Rate limits on signup, login, demo, claim and verification; login refused until a claimed address is verified; identical responses from resend-verification whether or not the address exists, so it cannot be used to enumerate accounts.
A08 Data integrity	Socket payloads are validated and length-capped; a <code>join-room</code> with no argument previously threw inside an async listener and told the client nothing, which is exactly the kind of silent failure that hides a worse bug.
A09 Logging and monitoring	One log line per completed request and one per failure, a request id echoed in the <code>X-Request-Id</code> header and in every error body, credential-shaped field names redacted on the way in, stacks kept out of responses, and no mail link logged in production where it would be a working credential.
A10 Server-side request forgery	The only outbound calls are to the operator's own configured endpoints — the mail webhook and the AI provider. Neither URL is user-supplied.

Also worth stating plainly: CORS is not treated as an authorization boundary. It stops a page from reading a response; it does not stop a script from calling the API. Every route assumes the request is hostile.

Performance and scalability

- **Media latency is structurally bounded.** Audio and video take the direct peer path, so the server adds nothing to it. The only server involvement in a call is a few signalling messages at setup.
- **The mesh is the scaling limit, and it is documented rather than hidden.** Connections grow with the square of the room size. Rooms are groomed for 2-6 video peers, and an SFU is the named next step above that. A claim of 500 or 5,000 participants would be a checkbox that falls over in the demo.
- **Database access is index-driven where it matters.** Indexes exist for the sweep's three lookups, for resolving a room code, and — specifically — for verification, because spending a public email link must not be a collection scan. Each index is declared next to the comment explaining what would otherwise scan.
- **Rate limits are keyed where the cost is.** Summaries are limited per *user*, not per IP, because every call may reach a paid model. The two IP-keyed limits are only safe behind `trust proxy`, which is why that setting is part of the deployment checks.
- **Nothing was load-tested, and this report says so.** What exists instead: per-request timing in the logs, an external check that measures health-endpoint response time, and CI that publishes test counts and coverage. A load test is listed in the roadmap rather than implied by a number that was never measured.
- **Known unbounded path, stated:** chat history is appended without a cap, so a meeting that ran for hours would grow its document without limit. The fix (a sliced push with a retention window) is recorded as outstanding work rather than left to be discovered.

Challenges faced and how they were solved

Challenge	How it was solved
A guest room per visitor would grow the database without limit	A retention sweep deletes guests and the rooms they own by age, a hard count caps how many guest rooms exist at once, and — the important part — a session with a connected socket is never cut off mid-call. The sweep runs <i>inside</i> the API process because deciding whether a demo is still running needs live socket membership, which a separate cron process cannot see.

Screen sharing broke the camera button	The camera toggle read the first video track of the outgoing stream, which during a share <i>is</i> the screen. The camera track is now held separately, and the screen track is swapped in with <code>replaceTrack</code> — no renegotiation, no second stream.
A screen share looked like a webcam to everyone else	A remote track carries no display surface, so peers cannot tell a slide from a face. A small <code>media-state</code> event now carries mic, camera and sharing flags, and everyone re-announces when someone joins so a newcomer's tiles are right immediately.
Two peers behind strict NATs could not connect at all	TURN support, plus tooling to prove it: a command that talks TURN directly and reports reachability, the credential challenge and a successful relay allocation, and a local coturn harness so the relay path can be exercised without an account. The credential realm and the allocation are the two things that silently fail.
A verification link is a working credential	Verification mail is never sent in production without a configured webhook, because a link in a production log is a credential anyone with log access can spend. Only the token hash is stored, so a database leak yields no usable links.
A wildcard CORS origin silently allowed nothing	The wildcard is stripped rather than honoured and reported at boot. A browser refuses a wildcard on a credentialed request, so honouring it would have been the hardest possible failure to read.
Every visitor behind a platform proxy shared one rate-limit budget	<code>trust-proxy</code> defaults to one trusted hop in production, which is exactly what a single platform proxy is, and a boot-time warning fires when it is off behind a proxy. Without it, about thirty demo clicks in fifteen minutes would stop the demo path answering worldwide.
A body over the size limit answered 500	Failures are classified before they are reported: an oversized body is now a 413, and a CORS rejection names the setting. Chasing a bug that does not exist is the worst outcome of a wrong status code.
<code>tee</code> in CI reported green on a failing test suite	The pipeline reported the exit status of <code>tee</code> , which is always zero. <code>set -o pipefail</code> fixes it, and a test now reads the workflow and fails if any capturing step drops the flag — because that failure mode is invisible.
The CI coverage number was only readable with an authenticated log download	Each test job publishes its counts, failing test names and coverage beside the floor that enforces it to the run summary page, written even on a red run. The parser reads the floors from the configuration that enforces them, so the summary cannot disagree with the gate.
Free-tier hosting sleeps	A scheduled workflow pings the health endpoint every ten minutes, because a cold start of about fifty seconds would fail the load target outright.

7. Deployment and Operations

Platforms

Component	Platform	Notes
API and Socket.io	Render (free tier)	<code>render.yaml</code> describes the whole service shape including environment variables, so the deploy is a blueprint import rather than a settings tour. Rendered over HTTPS automatically.
Client	Vercel	<code>vercel.json</code> supplies the build command, output directory and the SPA rewrite that makes <code>/room/<code></code> work on refresh.
Database	MongoDB Atlas M0	A free cluster; the connection string is an environment variable. The API reaches it over the standard driver with TLS.
Relay (optional)	Any TURN server, or coturn locally	Only needed for peers behind strict or symmetric NATs. The repository carries a Compose file for a local coturn so the relay path can be proven without an account.
CI/CD	GitHub Actions	Two workflows: the test pipeline on every push and pull request, and the keep-alive ping on a schedule.
AI provider (optional)	OpenAI	Absent by default, and the application is fully functional without it.

Configuration is explicit and checked

Every setting lives in one of two example files, and the difference between them is a documented one: the server's file and the client's file each carry a comment explaining what every variable does, including the ones that default sensibly. The production-shaped surface is small — a connection string, a signing secret, the allowed client origin, the public app URL — and the deployment checks in `config/deployment.js` report at boot the four mistakes that are invisible from inside the process:

- a placeholder or short JWT secret (anyone who guesses it can mint a token for any account);
- an origin that still says localhost (the deployed client then fails with an opaque CORS error);
- verification enabled with no mail transport (a claimed account could never be verified, so login stays refused forever);
- `trust proxy` off behind a platform proxy (one rate-limit budget shared by the entire internet).

These are logged once at boot, where deployment output is read, and exposed to the operator through the admin stats endpoint. They are deliberately *not* on the public health endpoint, because "JWT_SECRET is not set" is an invitation, not a status code. None of them is fatal: a health check that fails takes the whole demo down, which is worse than the misconfiguration it reports.

CI/CD pipeline

Four jobs, none of which needs a database, Docker or a paid runner:

Job	What it does
server	Lints, then runs the suites under coverage with enforced floors (85% lines and functions, 70% branches).
client	Lints, runs the Vitest suite with its own thresholds (95% lines, statements and functions; 90% branches), then builds the production bundle.
format	Runs the Prettier check over both packages and the tooling directory with one root config.
scripts	Runs the tooling's own tests — the STUN codec against the published RFC test vectors, the log-summary parser, the report renderer (including the layout invariants that a PDF cannot check for itself) and the environment-parity check — with no install step at all , because they have no dependencies.

Each test job captures its output and then publishes a Markdown summary of it — test count, failures by name, and coverage next to the floor that enforces it — to the run summary page. That detail has a concrete reason: GitHub serves job *logs* only to an authenticated request, so without the summary a reader with the link cannot see the numbers. It is written on a failing run too, which is exactly when someone is looking. The pipeline doubles as the keep-alive host, so no external monitoring service is involved.

Monitoring and health

- `GET /api/health` — a heartbeat, and the endpoint the keep-alive ping hits.
- `GET /api/admin/stats` — retention configuration, who is connected right now, how many guest accounts exist against the cap, the last sweep's result, and the log level, format and proxy setting this process is using. Behind a token, absent (404) when the token is unset.
- Structured logging — JSON in production for a platform's log viewer, readable in

development, with a level switch and credential-shaped fields redacted.

- `npm run verify:demo` — an external check of a deployment: health and how long it took, a browser-shaped CORS preflight from the client's origin, one click of the demo, the room it landed in, a second visitor joining by link, and a refresh on a room URL to prove the SPA rewrite.

8. Visuals

Honest status rather than a gallery of placeholders: the screenshot set requires the deployed URL, because the point of a screenshot is to show the product as a visitor meets it. What exists today and what is planned:

In this document and the repository

1. The architecture diagram in section 4, drawn as text because the submission's own guidelines allow clean ASCII art where no drawing tool was used, and because a diagram that lives in the source control diff is one that stays true. A Mermaid version of the same diagram ships in `docs/architecture.md` and renders natively on the repository page.
2. The screenshots below are a defined shot list rather than an aspiration: each has the exact state to capture, so the set is reproducible rather than whatever happened to be on screen.

The shot list to capture against the live demo

#	Capture	Why it is the frame worth keeping
1	Landing page, one click to demo	The highest-weighted rubric item, visible in a single frame with no form filled in
2	Room with two participants and live tiles	Proves the multi-user session: two names, two cameras, one room
3	Screen share active, with the sharing flag on the tile	The feature that is hardest to fake in a still image
4	Chat with a message from each participant	Real-time chat persisted, not echoed locally
5	AI summary and action items, one item ticked	The product's differentiator, and the tick is shared state
6	Dashboard history with the search box filtering	The post-meeting half of the product
7	Exported Markdown document	The artifact a team would actually keep
8	CI run page with the published summary visible	The numbers a reviewer can check without a token
9	TURN-relayed call with the amber badge	Honest reporting of which network path was used
10	Lighthouse report on the deployed URL	Initial-load and accessibility figures, measured rather than claimed

Motion, where a still frame cannot carry the point — a tick propagating to the second participant, a mute labelling the peer's tile, a screen share replacing the camera — belongs in the demo video rather than in a screenshot, and the demo runbook in the repository scripts exactly those beats with timings.

9. Personal Reflection

Key learnings

Scope cuts are architecture, not omission. The decision that shaped this project was not to build a media server. The specification asks for 500 to 5,000 concurrent participants; a mesh peer-to-peer topology serves six well and fails gracefully above that. Choosing the smaller topology made the difference between a product that runs on a free instance and can be demoed from any laptop, and one that needs infrastructure nobody grading it would stand up. The same logic applied to transcription: rather than half-build a paid feature, the summary path was built so that it produces real output with no provider at all.

A demo path is a feature with its own requirements. The one-click guest turned out to touch authentication, session issuing, data seeding, retention, count bounding, and the dashboard, because a visitor who arrives with no account still has to own something, and whatever they own has to be cleanable. It also produced the best invariant in the project: a demo that is still in progress is never swept, which meant the cleanup had to live inside the process that can see who is connected, rather than in a cron job beside it.

Verification beats assertion, every time. Nearly every serious bug found here was found by running the real thing rather than a fixture of it: the summary parser looked correct against hand-written fixtures and broke against real CI output, because the test runner colours its output even when piped to a file and a duration line is a float, not an integer. A command that reported success while the pipeline was failing was found by testing the failing case deliberately.

The invisible failure is the one worth engineering against. The wildcard origin, the missing `pipefail`, the camera toggle that read the screen track, the socket payload with no argument: none of these produces an error message that points at the cause. They produce an empty tile, a green pipeline, a muted presentation, and silence. The pattern that emerged was to make each one either impossible or loud — strip the wildcard and warn, pin the pipeline flag with a test, guard the payload and answer with a reason.

Free-tier reality is a design constraint, not an afterthought. A sleeping instance takes about fifty seconds to answer, which directly contradicts a five-second load target, so a scheduled ping is not a nice-to-have but part of the product. Similarly, in-memory rate-limit stores are correct for one instance and wrong for two, and saying so is better than discovering it in production.

Industry best practices applied

- Conventional commits with a documented type table, and a history where every commit message explains *why* rather than restating the diff.
- One root formatting configuration rather than one per package, with the check enforced in CI so it cannot drift.
- A coverage floor enforced by the test command itself, so a falling number fails the build instead of being noticed in a report nobody opens.
- Tests that run without external services: the database is stubbed, so the suite is fast, deterministic and runnable anywhere, including in CI with no service containers.
- Dependency hygiene: the server's dependency tree is audited clean, new dependencies are argued for rather than added, and the tooling that could have been a library (the PDF renderer, the log-summary parser, the STUN codec check) is written against the standard library instead.
- Configuration as documentation: both example environment files explain every variable and

its default, and configuration errors are reported at boot rather than left to a visitor to discover.

- Security posture stated as specific, checkable properties rather than adjectives — which caps exist, where they are enforced, and what each one is defending.

Future roadmap

1. **Recorded meetings** — a canvas-composited `MediaRecorder` producing a downloadable WebM, which needs no server and no account.
2. **Live transcription** — the browser's own speech recognition is free and requires no key, which is what makes it compatible with this project's rule that a judge must be able to run everything. The specification's accuracy target should not be claimed for it; it should be measured and reported.
3. **Team workspaces and a Kanban board** — the largest remaining gap against the specification, and the natural consumer of the action items this build already extracts reliably. The data shape is already right; what is missing is the board.
4. **Analytics** — meetings held and attended, duration, items completed, drawn as plain SVG. The interesting part is not the charts but defining a metric that is not vanity.
5. **A media server for larger rooms** — LiveKit or mediasoup, which is the point at which the mesh's square-growth limit stops being a design decision and starts being a ceiling.
6. **Multi-instance readiness** — a shared adapter for Socket.io and a shared rate-limit store, in that order, because the moderation path currently reaches sockets through an in-process reference and would not work across instances.
7. **Load testing** — a real concurrent-participant measurement, so the concurrency claim stops being a design statement and becomes a number.